

PROGRAMACIÓN DE MICROCONTROLADORES

Modelación y Máquinas de Estados Finitos

**Carrera de Especialización en Sistemas Embebidos
Facultad de Ingeniería - UBA**

¿Qué es un modelo?

- Un **modelo** es una **representación simplificada de un sistema** que contempla sus propiedades importantes desde un determinado punto de vista.
- Los modelos de mayor utilidad se caracterizan por ser:
 - **Abstractos.** Enfatizan los aspectos importantes del sistema y eliminan los aspectos irrelevantes.
 - **Comprensibles.** Expresados en forma fácilmente perceptible por los observadores.
 - **Precisos.** Representan fielmente el sistema modelado.
 - **Predictivos.** Sirven para responder cuestiones sobre el sistema modelado.
 - **Económicos.** Mucho más baratos de construir y estudiar que el propio Sistema.

¿Qué es un modelo?

- Los modelos empleados para crear software para sistemas embebidos, además de servir para **lograr un conocimiento más profundo del problema**, también se utilizan como elementos de entrada para la **generación de código**.
- La mayoría de los enfoques actuales en el desarrollo de software basado en modelos coinciden en:
 - Utilizar una **representación gráfica del sistema a desarrollar**.
 - Describir el sistema con un cierto grado de **abstracción**.
 - Generar **código ejecutable para el sistema embebido a partir del propio modelo**.

Máquina de Estados Finitos

- Un **modelo** altamente utilizado es el de **máquina de estados finitos**.
- Constituye una herramienta gráfica utilizada tradicionalmente para modelar el comportamiento de sistemas electrónicos e informáticos.
- Una MEF (o FSM en inglés) es un **modelo computacional**, basado en la teoría de autómatas, que se utiliza **para describir sistemas cuyo comportamiento depende de los eventos actuales y de los eventos que ocurrieron en el pasado**.
- En cada instante de tiempo la máquina se encuentra en un estado, y dependiendo de las entradas, actuales y pasadas, que provienen del ambiente, la máquina cambia o no cambia de estado, y puede realizar acciones que a su vez influyen en el ambiente.

Máquina de Estados Finitos

- El modelado puede hacerse utilizando un “diagrama de estados” o “tabla de estados”.
 - La máquina de estados va cambiando de estados según indiquen los “flags” de estado.
 - Existen reglas bien definidas para cambiar de estado.
 - Cada transición implica diferentes respuestas del sistema.
 - Existen dos tipos de implementaciones de MEF:
 - Moore.
 - Mealy.
- } Difieren en la forma en que se produce la salida.

Moore

- La **salida** del sistema depende del **estado actual**.
- El **siguiente estado** depende de la **entrada** y del **estado actual**.
- Puede haber **múltiples estados** con la **misma salida**, pero cada uno de ellos es un estado diferente.

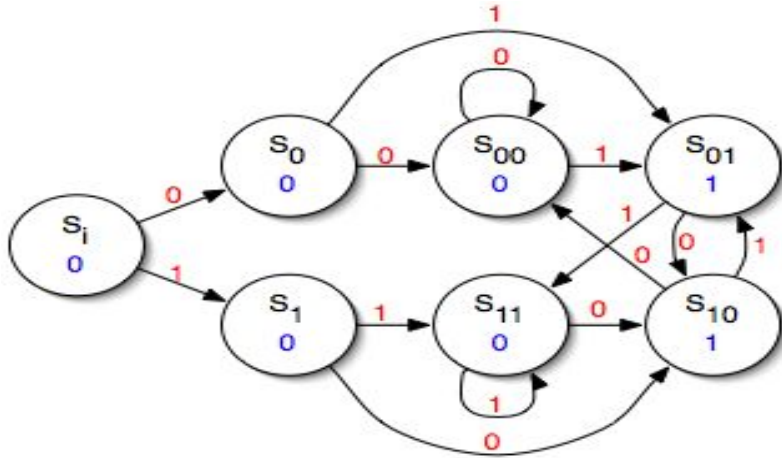
Mealy

- La salida depende del estado actual y la entrada.
- El siguiente estado depende de la entrada y del estado actual.
- Las diferentes salidas son necesarias para el cambio de estado.

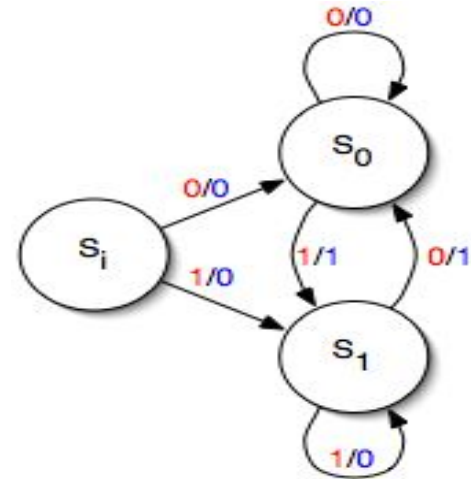
Ejemplo 1: detector de flanco

- La salida es la XOR de las dos entradas más recientes.

Moore:



Mealy:



Estados
Entradas
Salidas

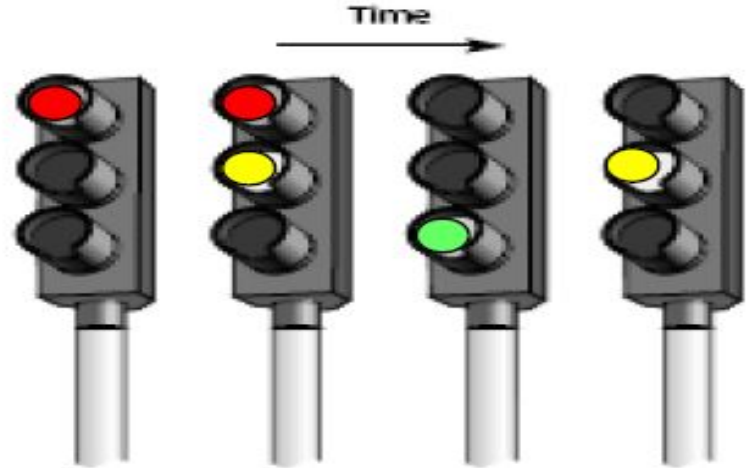
Transiciones de estado en una MEF

- **Tiempo:** depende solamente del paso del tiempo.
 - Ej: en el estado A la función a se ejecuta por x seg, luego se pasa al estado B en donde la función b se ejecuta por y seg y así sucesivamente.
- **Tiempo-entradas:** depende del tiempo y de las entradas.
 - Ej: el sistema se mueve del estado A al B si una entrada particular se recibe dentro de un intervalo de tiempo.
- **Entradas-Salidas:** la transición entre estados depende de las entradas (sensores) y las salidas (actuadores).

Ejemplo 2: semáforo

Estados:

- 1-rojo
- 2-rojo-amarillo
- 3-verde
- 4-amarillo

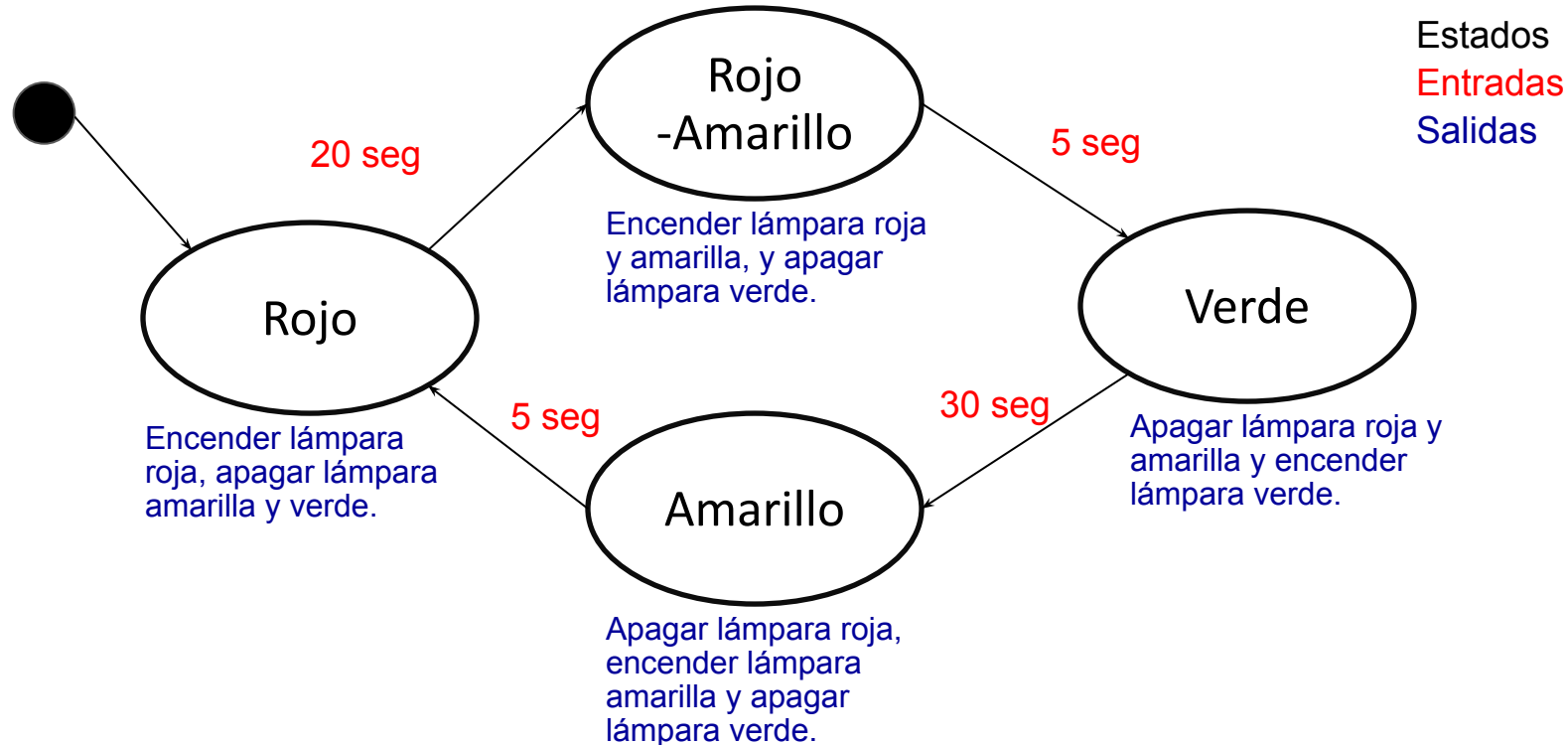


¿Qué tipo de MEF utilizaría para describirlo,
Moore o Mealy?

Es más adecuada la representación de Moore

- ¿Cuál es la entrada?
 - Tiempo transcurrido.
- Analicemos las posibles salidas del sistema:
 - En estado **Rojo**. Encender lámpara roja, apagar lámpara amarilla y verde.
 - En estado **Rojo-Amarillo**. Encender lámpara roja y amarilla, y apagar lámpara verde.
 - En estado **Amarillo**. Apagar lámpara roja y amarilla y encender lámpara verde.
 - En estado **Verde**. Apagar lámpara roja, encender lámpara amarilla y apagar lámpara verde.
- La salida depende únicamente del **estado actual**. *Condición de Moore.*
- Los cambios de estado dependen del **estado actual** y del **tiempo transcurrido (entrada)**. *Condición de Moore o Mealy.*
 - *Cumple ambas condiciones de Moore.*

Diagrama de Estados del ejemplo semáforo



Implementación en C

```
// Nuevo tipo de datos enumerado llamado estadoMEF_t
typedef enum{
    ESTADO_INICIAL,
    ESTADO_1,
    ESTADO_2,
    ESTADO_N
} estadoMEF_t;

// Variable de estado (global)
estadoMEF_t estadoActual;

// Prototipos de funciones
void inicializarMEF(void );
void actualizarMEF(void);
```

Implementación en C

```
/// Programa principal
int main (void){
    ...
    inicializarMEF();
    ...
    while(1){
        ...
        actualizarMEF();
        ...
    }
    return 0;
}

// Función Inicializar MEF
void inicializarMEF(void){
    estadoActual = ESTADO_INICIAL;
    // Resto de la inicializaciones
}
```

Implementación en C

```
// Función Actualizar MEF
void actualizarMEF(void){
    switch (estadoActual) {
        case ESTADO_INICIAL:
            // Actualizar salida del estado
            gpioWrite(...)
            ...
            // Chequear condiciones de transición de estado
            if(condicionesDeTransición == TRUE){
                // Cambiar a otro estado
                estadoActual = ESTADO_N;
            }

            break;
        case ESTADO_1:
            ...
            break;
```


Implementación en C

```
case ESTADO_N:
```

```
...
```

```
break;
```

```
default:
```

```
    //Si algo modificó la variable estadoActual  
    // a un estado no válido llevo la MEF a un  
    // lugar seguro, por ejemplo, la reinicio:  
    controlDeErrores();  
    inicializarMEF();
```

```
break;
```

```
}
```

```
}
```